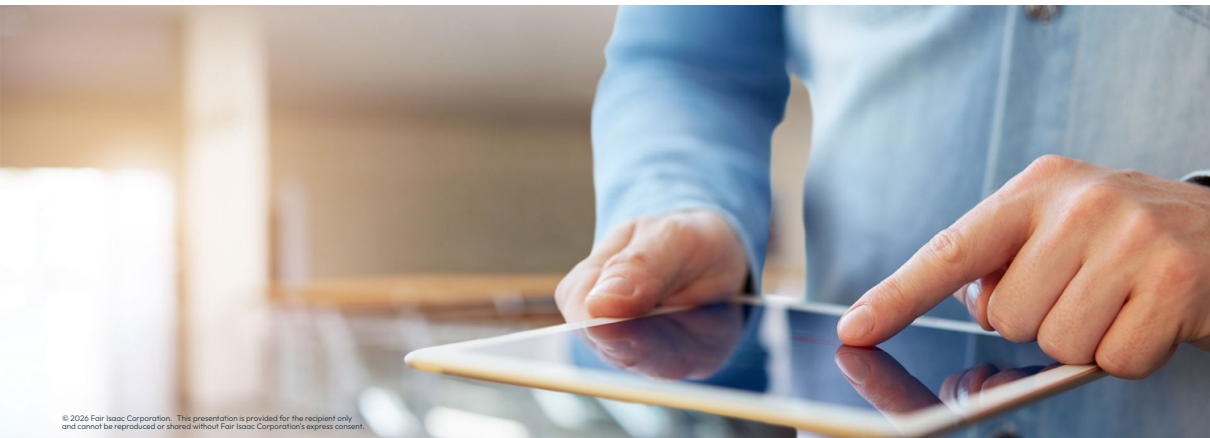




Introduction

Xpress Optimization, FICO



Topics

- Introduction to Xpress
- Modeling with Mosel:
 - Linear and Mixed Integer Programming (LP and MIP)
 - Accessing data sources
 - Programming language features
- Embedding models in applications

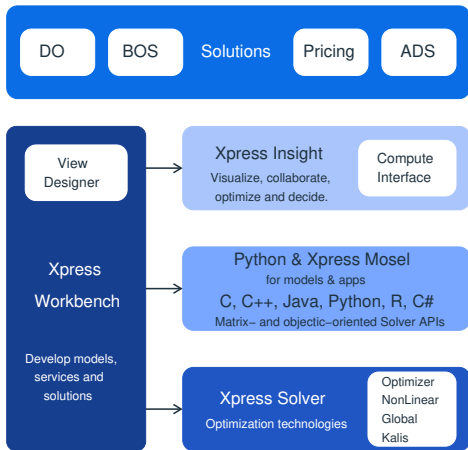


Overview of Xpress

- Xpress innovations
- Components of FICO Xpress

FICO Xpress Optimization

The most complete optimization software on the market



Xpress Solutions

- Point-and-click applications designed for business users, includes **Decision Optimizer** and **Business Outcome Simulation**.

Xpress Technology

- **Xpress Insight** – Rapidly deploy analytic & optimization models as powerful applications.
 - **Compute Interface** for optimization execution.
- **Python or Xpress Mosel** – develop models and apps in the preferred language.
- **Xpress Solver** – Optimization algorithms and technologies to solve linear, mixed integer, non-linear, global and constraint programming problems.
- **Xpress Workbench** – IDE for developing optimization models, services, and solutions.
 - Drag & Drop view configuration using **View Designer**

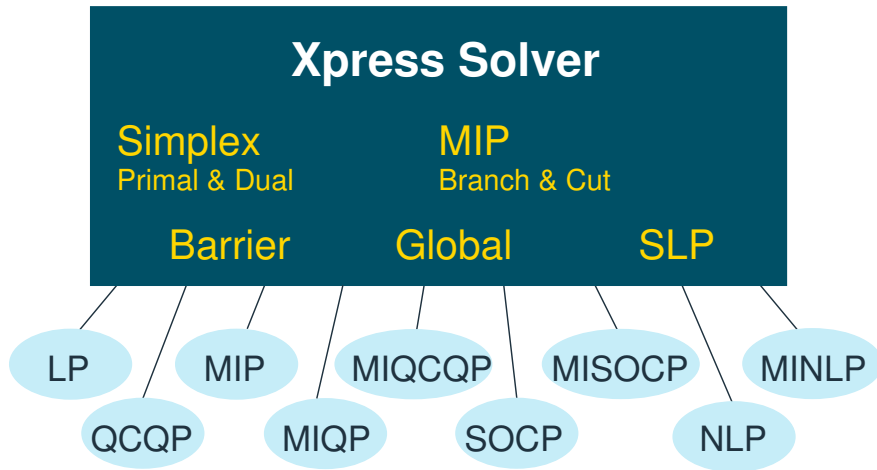
Overview of Xpress

- Optimization algorithms
 - solve different classes of problems
 - built for speed, robustness and scalability

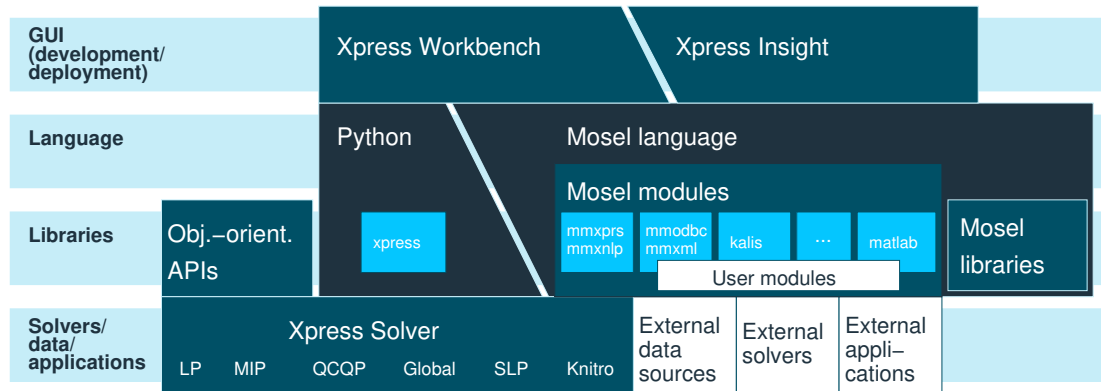
Overview of Xpress

- Optimization algorithms
 - solve different classes of problems
 - built for speed, robustness and scalability
- Modeling interfaces
 - Mosel
 - formulate model and develop optimization methods using Mosel language / environment
 - Python and object-oriented APIs
 - build up models in your application code using APIs for C, C++, Java, .NET, Python, R, Julia, MATLAB
- Application development
 - Xpress Insight
 - deploy multi-user optimization applications

Optimization algorithms



Overview of Xpress



43 years of business transformation with FICO Xpress Optimization

- Solving

1983: LP solver running on PCs

1992: parallel MIP (1997 on distributed PC/Linux networks)

1995/1996: commercial branch and cut algorithm

1998: bound switching in dual simplex

2003: lift-and-project cuts

2009: parallel MIP heuristics

2010: LP/MIP solver crosses 64-bit coefficient indexing threshold

2013: automatic solver selection for NLP

2014: parallel simplex

2016: task-based parallel MIP

2017: Barrier warm start and parallel crossover; parallel black box optimization

2018: LP folding

2019: iterative solution refiner

2020: Machine Learning for scaling; remote execution

2021: ML for cutting in tree search

2023: Global optimization solver

2025: PDHG; LP solving on GPU

43 years of business transformation with FICO Xpress Optimization

- Modeling

1983: general purpose algebraic modeling language (mp-model)

2001: algebraic modeling language combining modeling, solving and programming (Mosel)

2005: profiler and debugger for a modeling language

2005: user-controlled parallelism at the model level

2010: algebraic modeling language supporting distributed computing

2012: remote invocation without local installation (XPRD)

2014: parallel profiler and debugger; robust optimization; encryption

2015: model annotations and mosel doc

2016: native language support (XPRNLS), Unicode; remote invocation protocol

2017: cloud-based model and solution development

2018: Mosel free and extensible with new solvers

2019: dynamic package loading; namespaces; hashmap array type

2020: cloning of models and data sharing; constants of native types

2022: union types; reflection

43 years of business transformation with FICO Xpress Optimization

- Business user enablement

2012: Xpress Insight first released

2015: VDL for GUI definition

2016: Xpress Workbench for end-to-end optimization app development

2017-18: Pre-configured applications for Machine Learning (Pattern Discovery), ML-driven Forecasting

2019: View Designer for drag & drop UI configuration

2020: Compute Interface; Python-Insight app development

2022: Custom Progress Reporting

Our experience

Supply chain optimization

Portfolio generation + maintenance

Personnel scheduling

Production scheduling

Process industries (blending)

Vehicle routing

Marketing optimization

Generalized assignment (e.g. facility location)

Auctions

Airline operations

Electricity generation, supply and pricing

Mosel

- A modeling and solving environment
 - integration of modeling and solving
 - programming facilities
 - open, modular architecture
- Interfaces to external data sources (*e.g.* ODBC, host application) provided
- Language is concise, user friendly, high level
- Best choice for rapid development and deployment

Mosel: Components and interfaces

- **Mosel Native Interface (NI)**: to provide new or extend existing functionality of the Mosel language

⇒ *module (DSO)*

Modules of the Mosel distribution:

- *solvers*: mmxprs, mmquad, mmxnlp, kalis, nlsolv
- *data handling*: mmodbc, mmoci, mmsheet, mmxml
- *model handling, utilities*: mmjobs, mmsystem, mmhttp, mmssl
- *graphics, GUI*: mmsvg, mminsight

Mosel: Components and interfaces

- **Mosel Native Interface** (NI): to provide new or extend existing functionality of the Mosel language

⇒ *module (DSO)*

Modules of the Mosel distribution:

- *solvers*: mmxprs, mmquad, mmxnlp, kalis, nlsolv
- *data handling*: mmodbc, mmoci, mmsheet, mmxml
- *model handling, utilities*: mmjobs, mmsystem, mmhttp, mmssl
- *graphics, GUI*: mmsvg, mminsight
- **Xpress Workbench**: graphical user interface
 - model editor, debugging, deployment to Xpress Insight

Mosel: Components and interfaces

- **Mosel Native Interface** (NI): to provide new or extend existing functionality of the Mosel language

⇒ *module (DSO)*

Modules of the Mosel distribution:

- *solvers*: mmxprs, mmquad, mmxnlp, kalis, nlsolv
- *data handling*: mmodbc, mmoci, mmsheet, mmxml
- *model handling, utilities*: mmjobs, mmsystem, mmhttp, mmssl
- *graphics, GUI*: mmsvg, mminsight
- **Xpress Workbench**: graphical user interface
 - model editor, debugging, deployment to Xpress Insight
- **Visual Studio Code**: alternative editor with Mosel language support
 - syntax highlighting, run and debug Mosel models via the Xpress VS Code extension

Mosel: Components and interfaces

- **Mosel Native Interface** (NI): to provide new or extend existing functionality of the Mosel language

⇒ *module (DSO)*

Modules of the Mosel distribution:

- *solvers*: mmxprs, mmquad, mmxnlp, kalis, nlsolv
- *data handling*: mmodbc, mmoci, mmsheet, mmxml
- *model handling, utilities*: mmjobs, mmsystem, mmhttp, mmssl
- *graphics, GUI*: mmsvg, mminsight
- **Xpress Workbench**: graphical user interface
 - model editor, debugging, deployment to Xpress Insight
- **Visual Studio Code**: alternative editor with Mosel language support
 - syntax highlighting, run and debug Mosel models via the Xpress VS Code extension
- **Tools**: debugger, profiler, *mosel/doc*

FICO Xpress Mosel extension for Visual Studio Code

- Install from the **VS Code Marketplace**: search **FICO Xpress Mosel (BETA)**
- Activates automatically when a `.mos` file is open
- Language features: syntax highlighting, code folding, jump to definition, code completion, inline help, real-time diagnostics

```
13  model Prime
27
28  n:= 2
29  repeat
30  | while (not (n in SNumbers)) n+=1
31  | SPrime+={ n } ! n is a prime number
32
33  i:= n
34  while (i<=LIMIT) do ! Remove n and all its multiples
35  | SNumbers-={ i }
36  | i+=n
37  end-do
38  until SNumbers = { }
```

Expression: `n > 20`

Mosel and Optimizer Consoles

- Stand-alone command line executables with text interfaces
- Useful for simple deployment using batch/script files
- Available for all platforms supported by Xpress



Xpress Workbench demonstration

Xpress Workbench demonstration

- A tool for creating, editing, and debugging Xpress Insight apps (implemented with Mosel or Python) and Xpress Mosel models
- Insight Workbench enables you to do the following tasks:
 - create and edit source files (Mosel models, Python programs and VDL views)
 - publish Xpress Insight apps to an Insight server
 - debug an Insight scenario while it is executing within an Xpress Insight server
 - build an Xpress Insight app archive
 - debug a Mosel model in a standalone instance of Mosel
 - compile a Mosel model into a BIM file

Creating an Xpress Mosel project

FICO® Xpress Workbench

FICO.comCommunityHELP ▾

Recent projects

Create project

Open existing file or folder

Examples

- Introductory examples
- Insight examples
- Nonlinear examples
- Calling R from Mosel
- Calling Python from Mosel
- Browse all examples

Xpress Documentation

Ask the Xpress community

Choose from the options below. You will be asked to select a folder location in which the project files will be created.

Create an Xpress Mosel project

The project will contain a simple Mosel source file template. Choose this option if you want to develop a new Mosel model which is not intended to run within Xpress Insight.

CREATE MOSEL PROJECT

Create an Xpress Insight (Mosel) project

The project will contain templates for a Mosel model, custom views and companion file. Choose this option if you want to develop a new Xpress Insight app in Mosel.

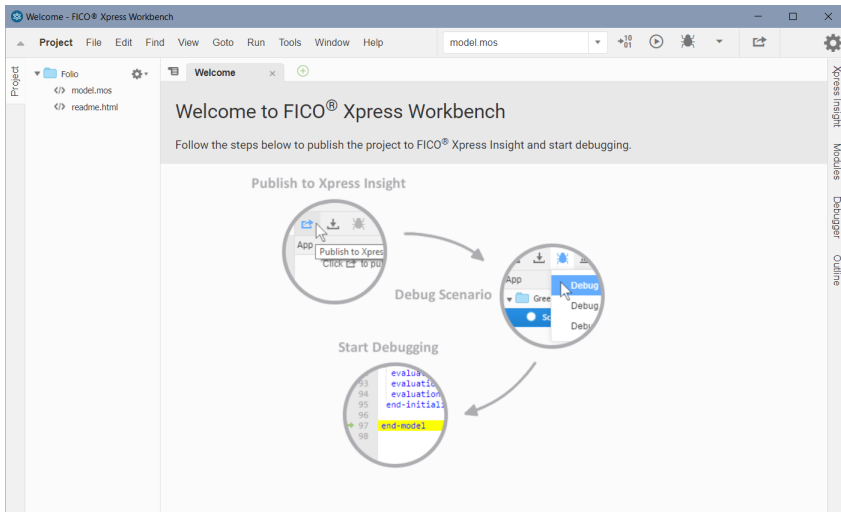
CREATE INSIGHT (MOSEL) PROJECT

Create an Xpress Insight (Python) project

The project will contain templates for a Python model, custom views and companion file. Choose this option if you want to develop a new Xpress Insight app in Python.

CREATE INSIGHT (PYTHON) PROJECT

Exploring the Workbench workspace



Xpress Workbench demonstration

- Workbench panes
 - model editor (central window)
 - project directory navigation and command history (left)
 - model output and execution log information (bottom)
 - debugging, deployment and collaboration information (right)
 -  Workspace preferences (settings).
 -  Toggle full-screen view for logging pane.

Xpress Workbench demonstration

- Editor

- code folding and breakpoint markers appear in the grey area immediately left to the text



Open a new file/tab



Subdivide and re-arrange panes in the editor window



Code folding for blocks of Mosel statements



Unfold folded code



Line position markers during debugging

Xpress Workbench demonstration

- Model execution
 - the name of the model is selected in the box next to these buttons, it may be different from the model(s) opened in the editor
 -  Compile a model.
 -  Execute (compile/load/run) a model.
 -  Execute a model in debug mode.
 -  Open Compiler Options or Run Dialog.
 - alternatively, use menu *Run* to compile or run a model

Debugging an Xpress Mosel project

The screenshot displays the FICO Xpress Workbench interface for debugging a Mosel project. The main editor shows the code for `foliopl.mos`, which includes constraints for risk, minimum values, capital spending, and investment bounds, followed by a maximization of the return and solution printing.

The bottom panel shows the command window output:

```
Compiling foliopl.mos to out\foliopl.bim with -G
Running model
Debug session 98f11d306a1d75c7fb52eeb418456ae2 started
4: 0%
5: 6.66666667%
6: 30%
7: 0%
8: 0%
9: 13.33333333%
10: 0%
The model is suspended after execution
```

The right sidebar contains the 'Xpress Insight' and 'Debugger' panels. The 'Debugger' panel shows the current state of the model, including the call stack and the variables table.

Call Stack:

Function	File	Ln	Col
(Model ended)	foliopl.mos	52	2

Variables:

Variable	Value	Type
Globals		Globals
frac	(10 entries) array of m...	array of m...
NA	(4 entries) set of inte...	set of inte...
entry	1	integer
entry	2	integer
entry	3	integer
entry	4	integer

Inspect in table:

Variable	Value	Type
Return	14.066667	lintr
RET	(10 entries) array of real	array of real
1	5	real
2	17	real
3	26	real
4	12	real
5	8	real
6	9	real
7	7	real

Xpress Workbench demonstration

- Debugger
 - breakpoints are set by clicking onto the gray area (left to the line number if it is displayed) preceding each row in the editor window
 - breakpoint conditions can be added via the right mouse button menu on the breakpoint icon
 - ○ Delete breakpoint/desactivated breakpoint.
 - ○ Delete a conditional/desactivated conditional breakpoint.

Xpress Workbench demonstration

- Navigating in the debugger
 - buttons are activated during debug runs
 -  Activate/deactivate all breakpoints.
 -  Start/stop the debugger.
 -  Resume/suspend model execution.
 -  Step over an expression.
 -  Step into an expression.
 -  Step out of an expression.
 -  Don't pause on exceptions.

Xpress Workbench demonstration

- Deployment to Xpress Insight
 - only if Xpress Insight has been installed
 -  Publish selected model to Insight.
 -  Build an Insight app archive.
 -  Debug a scenario.
 -  Edit Tableau workbooks.
 -  Refresh Insight scenario tree.

Project work [P-1]: Working with Workbench

- Start up Xpress Workbench
 - double click on a Mosel model, or
 - double click on the Workbench icon  on your desktop
- Execute a model and observe the changes to the displays in Workbench.
- Search for information on 'exportprob' and 'verbose' in the online help.

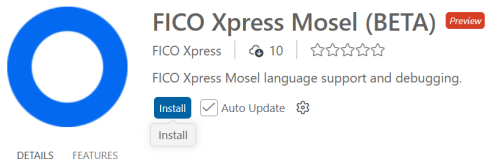
Reference material

- The manual [Getting Started with Xpress](#) introduces first time or occasional users to modeling with Mosel and BCL, or the direct Optimizer interface
- The [Evaluators Guide](#) and [Advanced Evaluators Guide](#) provide a quick walk-through of the Getting Started examples and some more advanced features
- The [Xpress Workbench Guide](#) explains how to edit models with this GUI

 VS Code Mosel extension

VS Code Mosel extension

- An extension for Visual Studio Code that brings full Mosel language support inside the editor
- Install from the VS Code Marketplace (Ctrl+Shift+X): search for **FICO Xpress Mosel (BETA)**
 - activates automatically when a .mos file is open
 - language features work without Xpress; runtime features require a valid Xpress installation
- Installing the Mosel extension



Syntax highlighting, code folding, and navigation

- **Syntax highlighting**: keywords, string literals, numeric constants, comments, and identifiers each receive distinct visual treatment

```
model Prime
  parameters
    LIMIT=100 .....! Search for prime numbers in 2..LIMIT
  end-parameters

  declarations
    SNumbers: set of integer .....! Set of numbers to be checked
    SPrime: set of integer .....! Set of prime numbers
  end-declarations

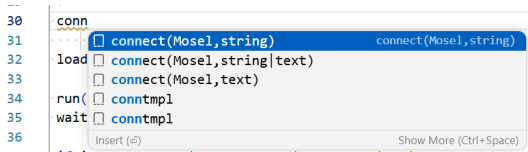
  SNumbers:= {2..LIMIT}

  writeln("Prime numbers between 2 and ", LIMIT, ":")
```

- **Code folding**: collapse and expand blocks such as parameters, declarations, initializations, procedure, and function
 - click the arrow in the editor gutter
- **Jump to definition**: press F12 on any symbol to jump to its declaration
 - works across files linked via uses, import, or include

Code intelligence

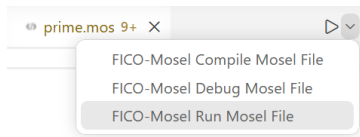
- **Code completion:** suggestions from the [Mosel Language Reference](#) and local declarations, with full signatures and documentation in the [detail panel](#)



- **Inline help:** hover over any symbol to see its declaration and documentation without leaving the current file
- **Real-time diagnostics:** errors and warnings appear inline from the Mosel parser, without triggering a full compile

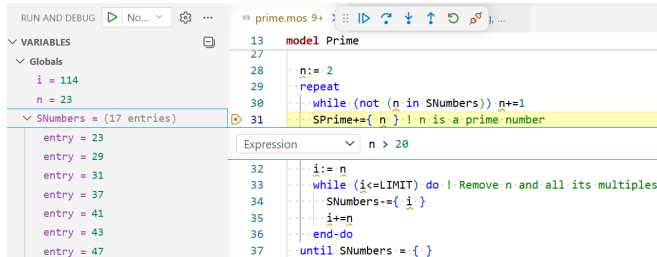
Runtime features

- A **play button** in the editor title bar opens a quick menu to compile, run, or debug the current `.mos` file
 - all commands are also available in the command palette (`Ctrl+Shift+P`)
- **Compile**: compile Mosel source files into portable binary (`.bim`) format
- **Run**: execute Mosel models with live output in a VS Code terminal
- **Debug**: full debugging support with breakpoints, variable inspection, and expression evaluation
- Runtime parameters can be passed via `.vscode/tasks.json` to automate compile and debug workflows



Debugging a Mosel model

- **Breakpoints**: set anywhere in Mosel code, including conditional breakpoints that trigger only when specific conditions are met
- **Variable inspection**: inspect variables in the Variables pane and the **Variable Inspector** panel, which expands sets and arrays into individual elements with their indices
- **Debug Console**: evaluate Mosel expressions and built-in functions against live model state, without modifying source code



Reference material

- The [Getting Started with Xpress](#) manual and the [Evaluators Guide](#) provide a quick introduction to modeling with Mosel
- The [Xpress Mosel Reference Manual](#) documents all language constructs and built-in functions supported by the extension
- The [FICO Xpress Mosel extension](#) is available on the VS Code Marketplace



<http://www.fico.com/xpress>